Laboratorio práctico - Operaciones CRUD con Node.js



Tiempo estimado necesario: 1 hora

En este laboratorio aprenderás a crear una **Lista de amigos** utilizando un servidor Express. Tu aplicación debe permitirte agregar un amigo con los siguientes detalles: `First name`, `Last name`, `Email` y `Date of birth`. También proporcionarás a la aplicación la capacidad de recuperar detalles, cambiar detalles y eliminar los detalles.

Estarás creando una aplicación con puntos finales de API para realizar operaciones de Crear, Recuperar, Actualizar y Eliminar sobre los datos mencionados utilizando un servidor Express.

También aprenderás a proporcionar acceso autenticado a los puntos finales. Utilizarás cURL y Postman para probar los puntos finales implementados.

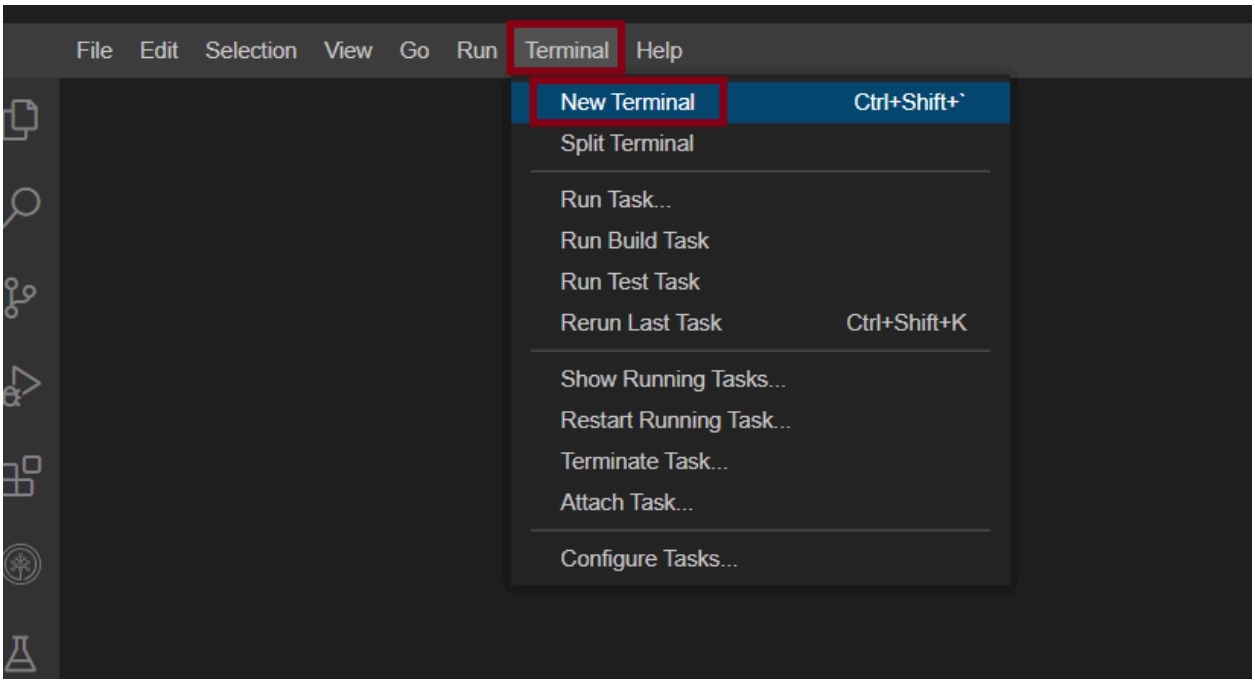
Objetivos:

- Crear puntos finales de API para realizar operaciones de Crear, Recuperar, Actualizar y Eliminar sobre datos transitorios con un servidor Express.
- Implementar autenticación a nivel de sesión utilizando JSON Web Tokens (JWT) para acceso autorizado.

The content of this lab is licensed under Apache 2.0

Configuración : Crear aplicación

1. Abre una ventana de terminal utilizando el menú en el editor: Terminal > Nueva Terminal.



2. Cambia a tu carpeta de proyecto, si aún no estás en la carpeta del proyecto.

```sh
cd /home/project
```

Run

3. Ejecuta el siguiente comando para clonar el repositorio git que contiene el código inicial necesario para este laboratorio, si aún no existe.

```sh
[ ! -d 'mxpfu-nodejsLabs' ] && git clone https://github.com/ibm-developer-ski
```

Run

The content of this lab is licensed under Apache 2.0

```
theia@theiadocke          /home/project$ [ ! -d 'mxpfu-nodejsLabs' ] && git clone https://github.com/ibm-developer-skills-network/mxpfu
-nodejsLabs.git
Cloning into 'mxpfu-nodejsLabs'...
remote: Enumerating objects: 100, done.
remote: Counting objects: 100% (47/47), done.
remote: Compressing objects: 100% (30/30), done.
remote: Total 100 (delta 28), reused 21 (delta 13), pack-reused 53
Receiving objects: 100% (100/100), 55.52 KiB | 5.55 MiB/s, done.
Resolving deltas: 100% (36/36), done.
theia@theiadocke          /home/project$
```

5. Cambia al directorio **mxpfu-nodejsLabs** para comenzar a trabajar en el laboratorio.

```sh
cd mxpfu-nodejsLabs/
```

Run

6. Enumera el contenido de este directorio para ver los artefactos de este laboratorio.

```sh
ls
```

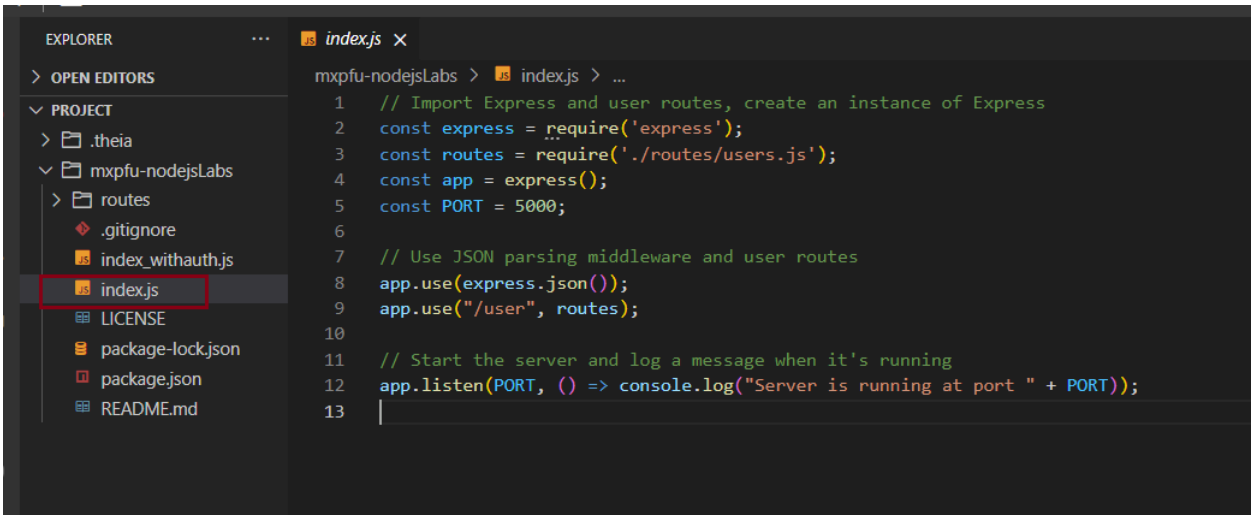
Run

```
theia@theiadocker-lavanyas:/home/project/mxpfu-nodejsLabs$ ls
index.js           LICENSE       package-lock.json  routes
index_withauth.js  package.json  README.md
```

The content of this lab is licensed under Apache 2.0

Ejercicio 1: Entender la aplicación del servidor y ejecutar el servidor

1. En el Explorador de Archivos, abre la carpeta **mxpfu-nodejsLabs** y visualiza **index.js**.



Tienes un servidor Express que ha sido configurado para ejecutarse en el puerto 5000. Cuando accedes al servidor con **/user**, puedes acceder a los endpoints definidos en **routes/users.js**.

Recuerda que GET, POST, PUT y DELETE son los métodos HTTP comúnmente utilizados para realizar operaciones CRUD. Estas operaciones recuperan y envían datos al servidor.

- `GET` se utiliza para solicitar datos de un recurso específico.
- `POST` se utiliza para enviar datos a un servidor para crear un recurso.
- `PUT` se utiliza para enviar datos a un servidor para actualizar un recurso.
- `DELETE` se utiliza para eliminar un recurso específico.

> *`POST` Y `PUT` a veces se utilizan de manera intercambiable.*

The content of this lab is licensed under Apache 2.0

2. Este laboratorio requiere que se instalen algunos paquetes. El paquete **express** y **nodemon** para iniciar y ejecutar el servidor Express y **jsonwebtoken** y **express-session** para la autenticación basada en sesiones.

- **express** - Esto es para crear un servidor que sirva los endpoints de la API.
- **nodemon** - Esto ayudará a reiniciar el servidor cuando realices cambios en el código.
- **jsonwebtoken** - Este paquete ayuda a generar un token web JSON que utilizaremos para la autenticación. Un **token web JSON (JWT)** es un objeto JSON utilizado para comunicar información de manera segura a través de Internet (entre dos partes). Se puede utilizar para el intercambio de información y se usa típicamente en sistemas de autenticación.
- **express-session** - Este paquete nos ayudará a mantener la autenticación para la sesión.

Estos paquetes están definidos como **dependencias** en packages.json.



json



```
  "dependencies": {
    "express": "^4.18.1",
    "express-session": "^1.17.3",
    "jsonwebtoken": "^8.5.1",
    "nodemon": "^2.0.19"
}
```

3. Observe que la aplicación express utiliza el middleware **express.json()** para manejar la solicitud como un objeto json.



plaintext



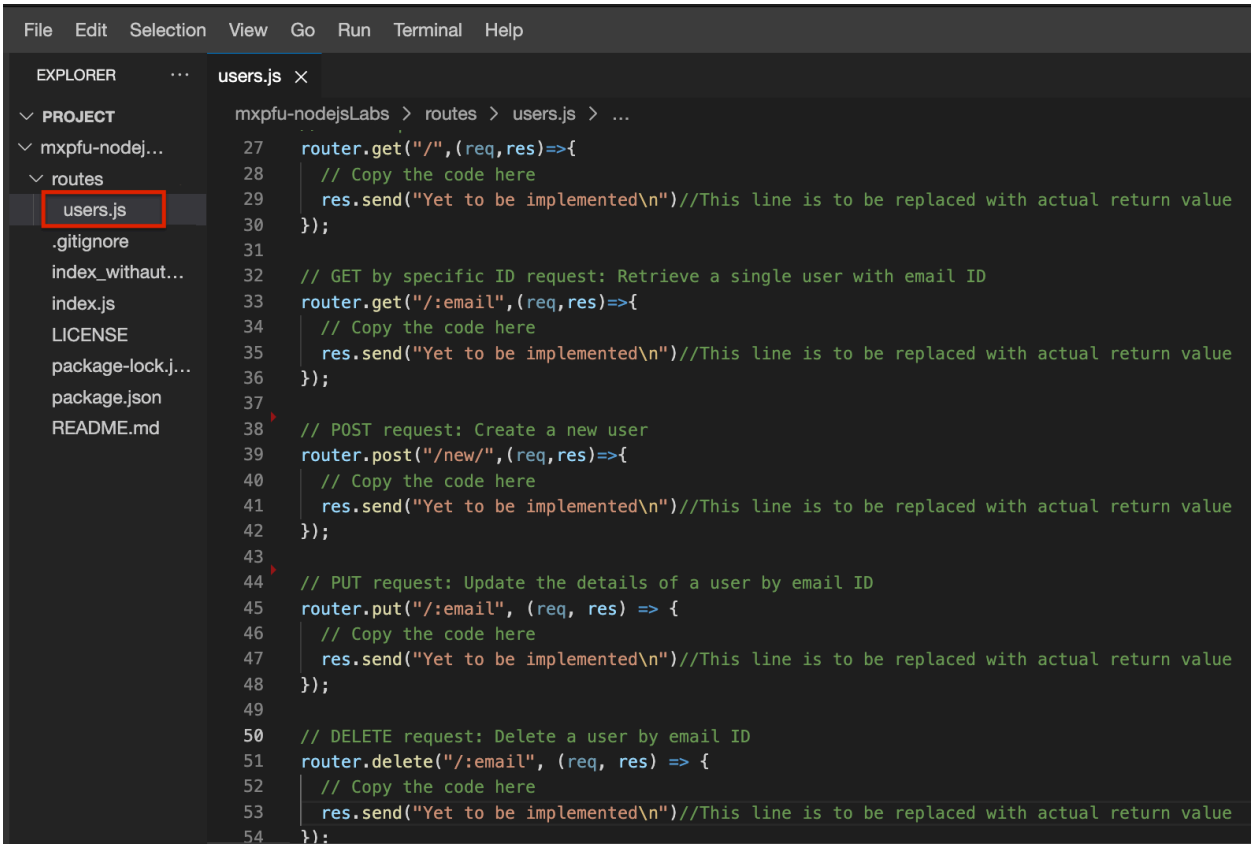
```
app.use(express.json());
```

4. Observe que la aplicación express utiliza rutas para manejar los endpoints que comienzan con **/user**. Esto significa que para todos los endpoints que comienzan con **/user**, el servidor irá a buscar un manejador de endpoints en **users.js**.

The content of this lab is licensed under Apache 2.0

```
app.use("/user", routes);
```

5. Todos los endpoints tienen una implementación esquelética, pero funcional en **users.js**. Navega a users.js en el directorio routes y observa los endpoints definidos en él.



Ejecutar el servidor

El código inicial proporcionado es un servidor en funcionamiento con valores de retorno de prueba. Antes de comenzar a implementar los endpoints reales, ejecuta el servidor.

1. En la terminal, imprime el directorio de trabajo para asegurarte de que estás en **/home/projects/mxpfu-nodejsLabs**.

The content of this lab is licensed under Apache 2.0

```sh
pwd
```

Run

2. Instala todos los paquetes necesarios para ejecutar el servidor. Copia, pega y ejecuta el siguiente comando.

```sh
npm install
```

Run

> *Esto instalará todos los paquetes requeridos según lo definido en packages.json.*

3. Inicia el servidor express.

```sh
npm start
```

Run

4. Abre un **Nuevo Terminal** desde el menú superior. Prueba un endpoint para recuperar estos usuarios. Esto aún no se ha implementado para devolver los usuarios.

```sh
curl localhost:5000/user
```

Run

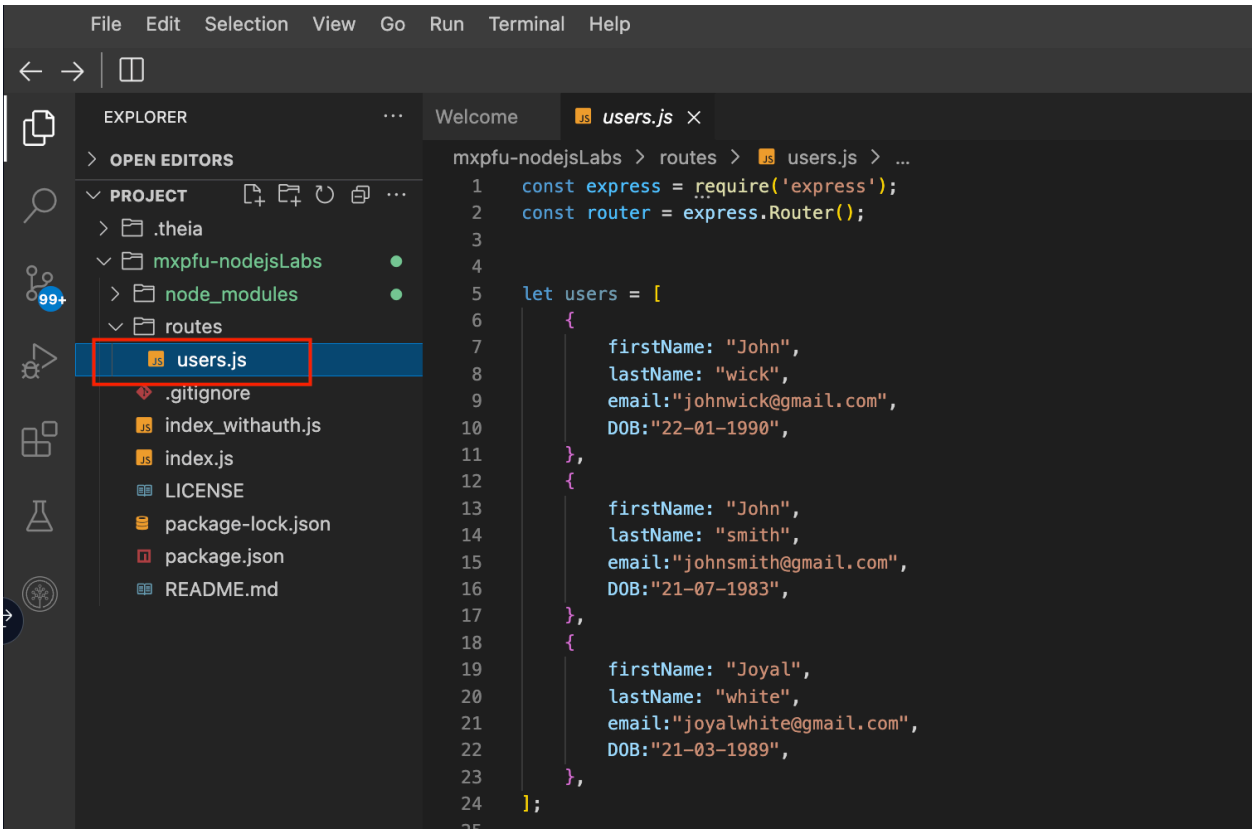
```
theia@theiadocker-lavanyas:/home/project$ curl localhost:5000/user
Yet to be implemented
```

5. Si ves la salida como se muestra arriba, significa que el servidor está funcionando como se esperaba.

The content of this lab is licensed under Apache 2.0

Ejercicio 2: Implementa tus endpoints

1. Navega al archivo llamado **users.js** en la carpeta **routes**. Los endpoints han sido definidos y se ha dejado espacio para que implementes los endpoints.

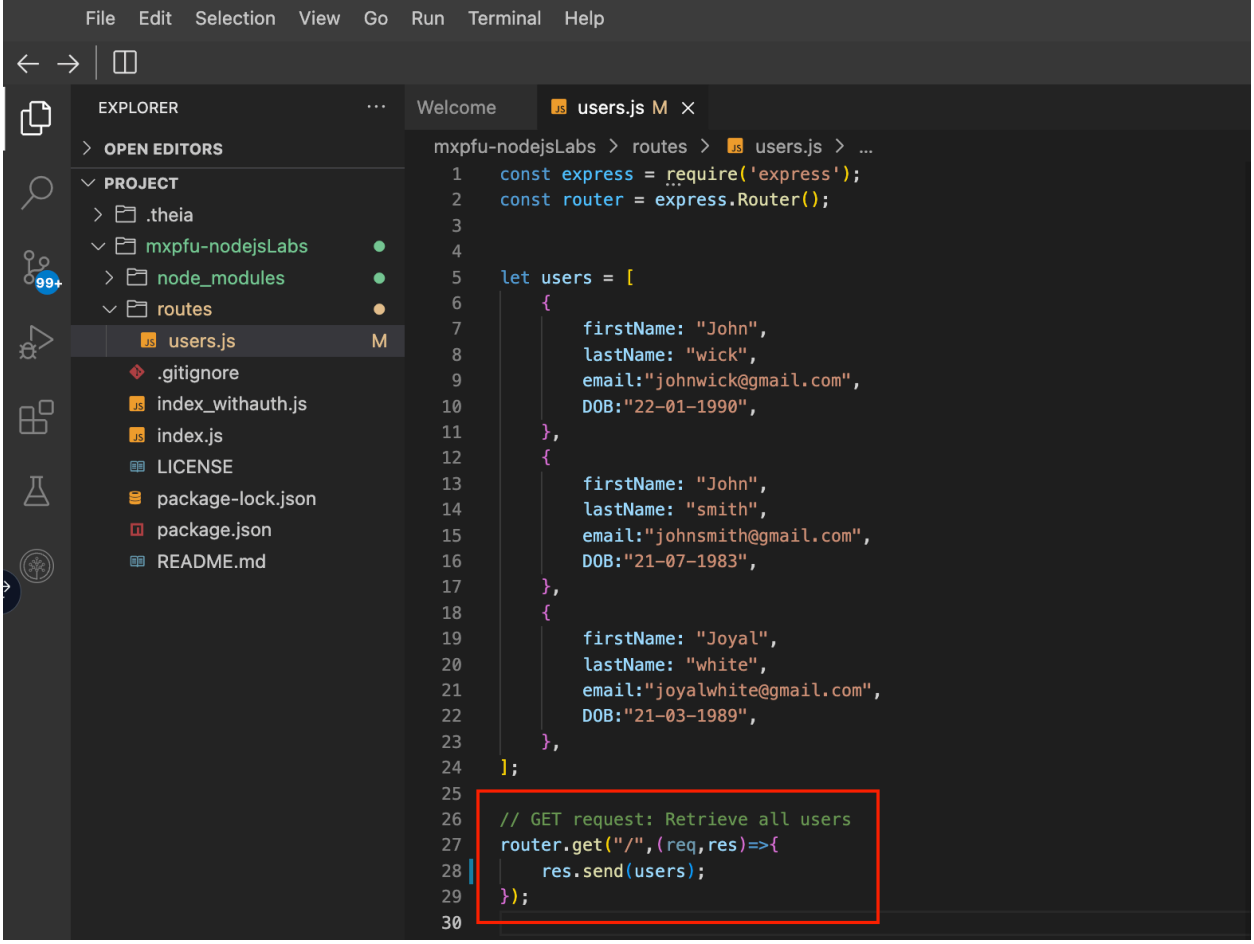


2. **R** en CRUD significa recuperar. Primero agregarás un endpoint de API, utilizando el método **get** para obtener los detalles de todos los usuarios. Se han añadido algunos usuarios en el código inicial.

- Copia el código a continuación y pégalo en users.js dentro de las llaves { } en el método **router.get("/",(req,res)=>{}**.

```
                                                                    js
```

The content of this lab is licensed under Apache 2.0



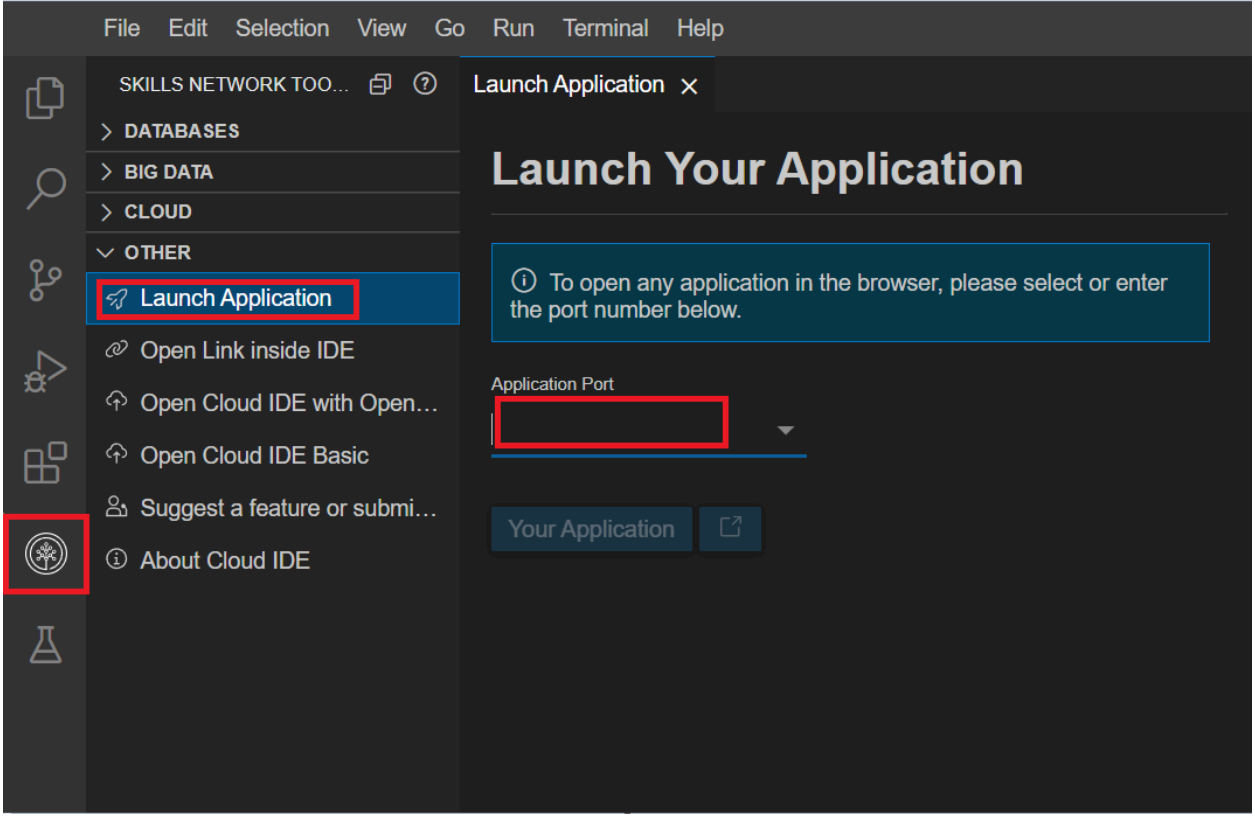
3. Asegúrate de que tu servidor esté en funcionamiento. A medida que realices cambios en el código, el servidor que iniciaste en la tarea anterior debe reiniciarse. Si el servidor no está en funcionamiento, inícialo nuevamente.

```sh
npm start
```

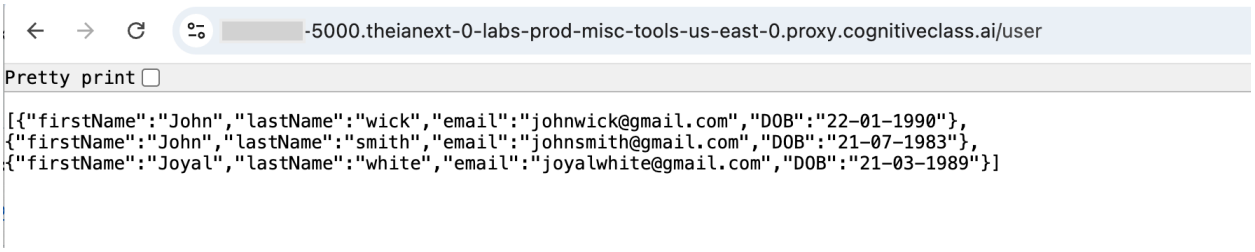
Run

3. Haz clic en el botón **Skills Network** a la izquierda. Esto abrirá la "Caja de herramientas de Skills Network". Luego haz clic en `OTHER` y luego en `Launch Application`. Desde allí deberías poder ingresar el puerto como `5000` y lanzar el servidor de desarrollo.

The content of this lab is licensed under Apache 2.0



4. Cuando se abra la página del navegador, añade **/user** al final de la URL en la barra de direcciones. Verás la página de abajo.



5. Verifica la salida de la solicitud GET utilizando el comando curl de la misma manera que lo hiciste en el ejercicio anterior.

```sh
curl localhost:5000/user/
```

Run

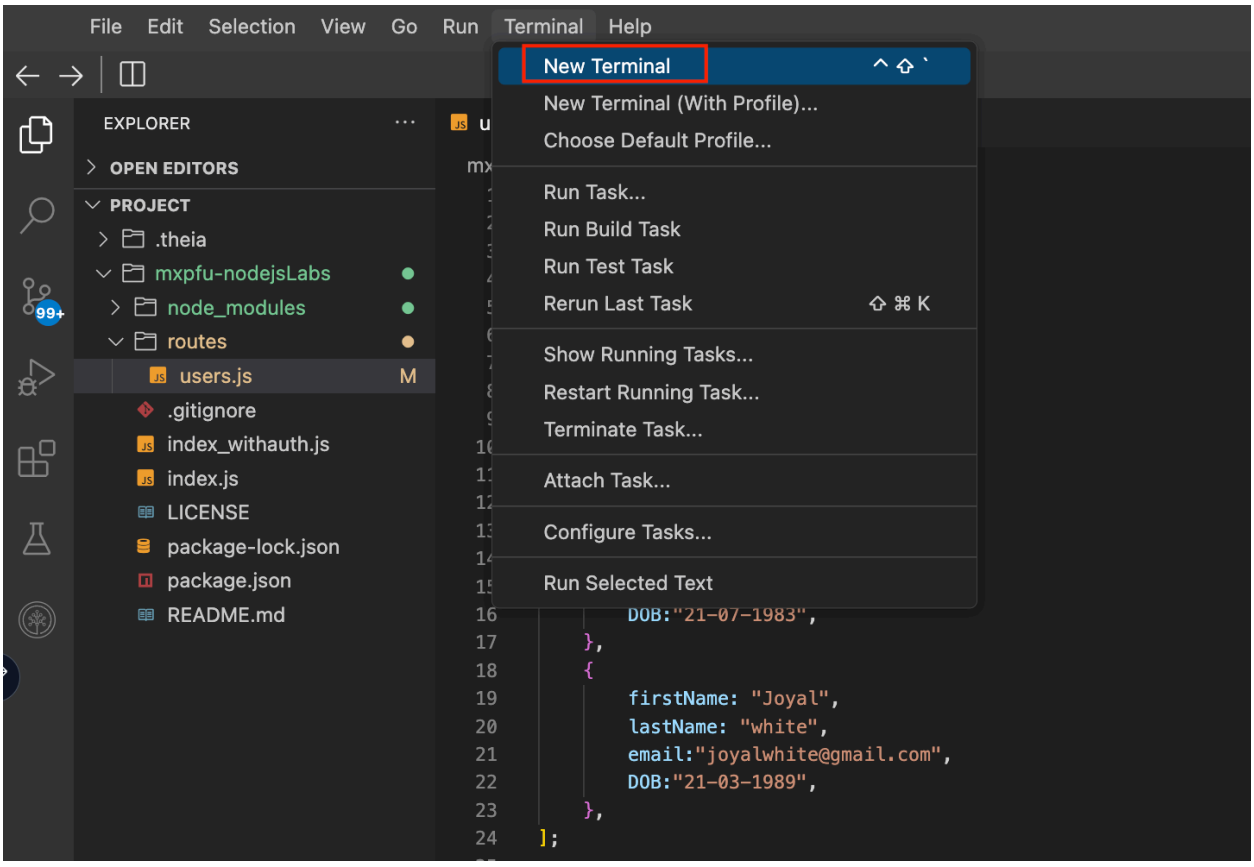
The content of this lab is licensed under Apache 2.0

Ejercicio 3: Creando un método GET por correo electrónico específico:

1. Implementa un método **get** para obtener los detalles de un usuario específico basado en su ID de correo electrónico utilizando el método `filter` en la colección de usuarios. Una vez que escribas el código y lo guardes, el servidor se reiniciará.

▶ Haz clic aquí para ver el código

2. Haz clic en Terminal > Nuevo Terminal



3. En el nuevo terminal, utiliza el siguiente comando para ver la salida del usuario con el correo electrónico johnsmith@gmail.com

The content of this lab is licensed under Apache 2.0

sh

```
curl localhost:5000/user/johnsmith@gmail.com
```

Run



The content of this lab is licensed under Apache 2.0

Ejercicio 4: Creando el método POST:

1. La **C** en CRUD significa **Crear**. Implementa el endpoint **/user** con el método POST para crear un usuario y añadirlo a la lista. Puedes crear el objeto usuario como un diccionario. Puedes usar el objeto usuario de ejemplo que se muestra a continuación.

```js
{
    "firstName":"Jon",
    "lastName":"Lovato",
    "email":"jonlovato@theworld.com",
    "DOB":"10/10/1995"
}
```

Utiliza `push` para agregar el diccionario a la lista de usuarios. Los detalles del usuario se pueden pasar como parámetros de consulta llamados *firstName, lastName, DOB y email*.

> *Consejo: El parámetro de consulta se puede recuperar del objeto de solicitud usando request.query.paramname*

▶ Haga clic aquí para ver el código

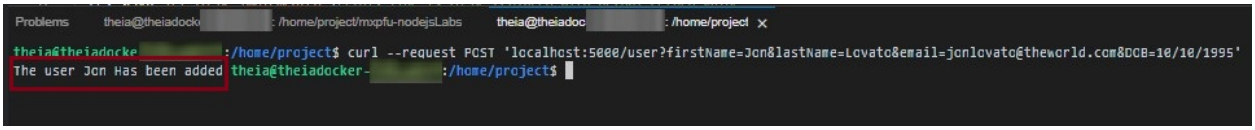
2. Utiliza el siguiente comando para publicar un nuevo usuario con el correo 'jonlovato@theworld.com' en la nueva terminal:

```sh
curl --request POST 'localhost:5000/user?firstName=Jon&lastName=Lovato&email=
```

Run

The content of this lab is licensed under Apache 2.0

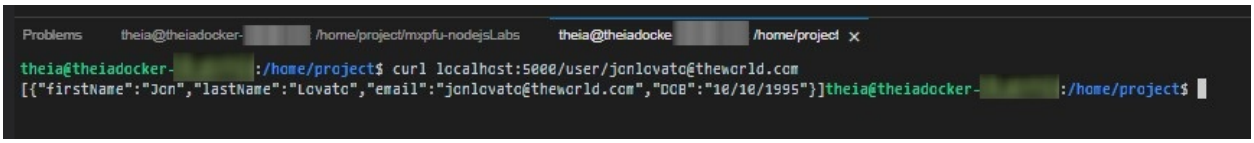
3. La salida será como se muestra a continuación:



4. Para verificar si el usuario con el correo electrónico 'jonlovato@theworld.com' ha sido agregado, puedes enviar una solicitud GET como se muestra a continuación:

```sh
curl localhost:5000/user/jonlovato@theworld.com
```

Run



The content of this lab is licensed under Apache 2.0

Ejercicio 5: Creando el método PUT:

1. La **U** en CRUD significa actualizar, lo cual se puede lograr utilizando el método PUT. Para realizar actualizaciones en los datos, utilizarás el método PUT. Primero debes buscar al usuario con el ID de correo electrónico especificado y luego modificarlo. El código a continuación muestra cómo se puede modificar la fecha de nacimiento (DOB) de un usuario. Realiza los cambios de código necesarios para permitir modificaciones en los otros atributos del usuario.

The content of this lab is licensed under Apache 2.0

```js
router.put("/:email", (req, res) => {
    // Extract email parameter and find users with matching email
    const email = req.params.email;
    let filtered_users = users.filter((user) => user.email === email);

    if (filtered_users.length > 0) {
        // Select the first matching user and update attributes if provided
        let filtered_user = filtered_users[0];

         // Extract and update DOB if provided

        let DOB = req.query.DOB;
        if (DOB) {
            filtered_user.DOB = DOB;
        }

        /*
        Include similar code here for updating other attributes as needed
        */

        // Replace old user entry with updated user
        users = users.filter((user) => user.email != email);
        users.push(filtered_user);

        // Send success message indicating the user has been updated
        res.send(`User with the email ${email} updated.`);
    } else {
        // Send error message if no user found
        res.send("Unable to find user!");
    }
});
```

2. El código completado se verá así.

The content of this lab is licensed under Apache 2.0

```
// PUT request: Update the details of a user by email ID
router.put("/:email", (req, res) => {
    // Extract email parameter and find users with matching email
    const email = req.params.email;
    let filtered_users = users.filter((user) => user.email === email);

    if (filtered_users.length > 0) {
        // Select the first matching user and update attributes if provided
        let filtered_user = filtered_users[0];

        // Extract and update DOB if provided
        let DOB = req.query.DOB;
        if (DOB) {
            filtered_user.DOB = DOB;
        }

        /*
        Include similar code here for updating other attributes as needed
        */
        // Extract and update firstName if provided
        let firstName = req.query.firstName;
        if (firstName) {
            filtered_user.firstName = firstName;
        }

        // Extract and update lastName if provided
        let lastName = req.query.lastName;
        if (lastName) {
            filtered_user.lastName = lastName;
        }

        // Replace old user entry with updated user
        users = users.filter((user) => user.email != email);
        users.push(filtered_user);

        // Send success message indicating the user has been updated
        res.send(`User with the email ${email} updated.`);
    } else {
        // Send error message if no user found
        res.send("Unable to find user!");
    }
});
```

3. Usa el siguiente comando para actualizar el `DOB` a `1/1/1971` para el usuario con el correo electrónico 'johnsmith@gmail.com' en la terminal dividida:

```
sh
curl --request PUT 'localhost:5000/user/johnsmith@gmail.com?DOB=1/1/197
```

Run

4. La salida será la siguiente:

```
theia@theia-nikeshkr:/home/project$ curl --request PUT 'localhost:5000/user/johnsmith@gmail.com?DOB=1/1/1971'
User with the email johnsmith@gmail.com updated.theia@theia-nikeshkr:
```

The content of this lab is licensed under Apache 2.0

5. Para verificar si la `DOB` del usuario con el correo electrónico 'johnsmith@gmail.com' ha sido actualizada, puedes enviar una solicitud GET como se muestra a continuación:

```sh
curl localhost:5000/user/johnsmith@gmail.com
```

Run

```
theia@theia-nikeshkr:/home/project$ curl localhost:5000/user/johnsmith@gmail.com
[{"firstName":"John","lastName":"smith","email":"johnsmith@gmail.com","DOB":"1/1/1971
theia@theia-nikeshkr:/home/project$
```

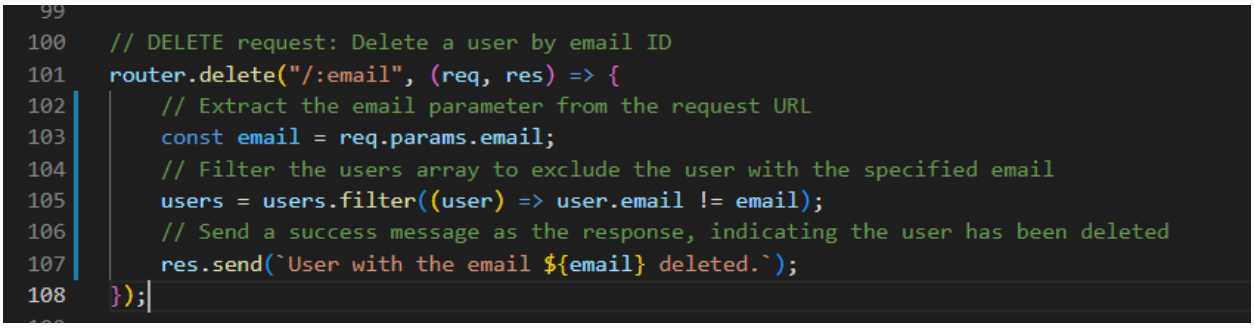
The content of this lab is licensed under Apache 2.0

Ejercicio 6: Creando el método DELETE:

1. La "D" en CRUD significa **Eliminar**. Implementa el método DELETE para eliminar el correo electrónico de un usuario específico utilizando el siguiente código:

```js
router.delete("/:email", (req, res) => {
    // Extract the email parameter from the request URL
    const email = req.params.email;
    // Filter the users array to exclude the user with the specified email
    users = users.filter((user) => user.email != email);
    // Send a success message as the response, indicating the user has been d
    res.send(`User with the email ${email} deleted.`);
});
```

2. El código completado se verá así.



3. Usa el siguiente comando para eliminar al usuario con el correo 'johnsmith@gmail.com' en la terminal dividida:

```sh
curl --request DELETE 'localhost:5000/user/johnsmith@gmail.com'
```

Run

The content of this lab is licensed under Apache 2.0

```
theia@theia-nikeshkr:/home/project$ curl --request DELETE 'localhost:5000/user
r/johnsmith@gmail.com'
User with the email johnsmith@gmail.com deleted.theia@theia-nikeshkr:/home/pr
oject$
```

5. Envía una solicitud GET para el usuario con el correo electrónico 'johnsmith@gmail.com' y asegúrate de que se devuelva un objeto nulo:

```
theia@theia-nikeshkr:/home/project$ curl localhost:5000/user/johnsmith@gmail.com
[]theia@theia-nikeshkr:/home/project$
```

The content of this lab is licensed under Apache 2.0

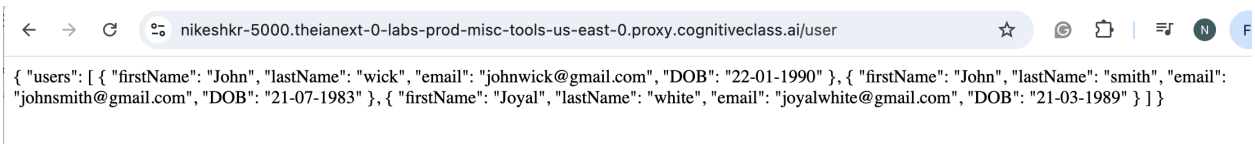
Ejercicio Opcional: Formatear la salida

1. Para hacer la salida más legible, puedes usar el método JSON stringify como se indica a continuación. Por favor, actualiza el código para el método GET a:

```js
// Define a route handler for GET requests to the root path "/"
router.get("/",(req,res)=>{
    // Send a JSON response containing the users array, formatted with an ind
    res.send(JSON.stringify({users}, null, 4));
});
```

2. Inicie la aplicación en el puerto 5000 y agregue 'user' al final de la URL.

3. Esto mostrará la salida del método GET como una cadena JSON según el método GET actualizado que se muestra a continuación:



The content of this lab is licensed under Apache 2.0

Ejercicio 7: Implementación de Autenticación

Todos estos puntos finales son accesibles por cualquier persona. Ahora verás cómo agregar autenticación a las operaciones CRUD. Este código ha sido implementado en **index_withauth.js**.

1. Observa el siguiente bloque de código en **index_withauth.js**.

```plaintext
app.use(session({secret:"fingerprint",resave: true, saveUninitialized: true})
```

Esto le indica a su aplicación express que use el middleware de sesión.

- **secret** - una clave única aleatoria utilizada para autenticar una sesión.
- **resave** - toma un valor booleano. Permite que la sesión se almacene de nuevo en el almacén de sesiones, incluso si la sesión nunca fue modificada durante la solicitud.
- **saveUninitialized** - esto permite que cualquier sesión no inicializada se envíe al almacén. Cuando se crea una sesión pero no se modifica, se denomina **no inicializada**.

El valor predeterminado tanto de **resave** como de **saveUninitialized** es verdadero, pero el predeterminado está en desuso. Por lo tanto, establezca el valor apropiado según el caso de uso.

2. Observe la implementación del endpoint **login**. Un usuario inicia sesión en el sistema proporcionando un nombre de usuario. Se genera un token de acceso que es válido por una hora. Puede observar esta longitud de validez especificada por **60 * 60**, que significa el tiempo en segundos. Este token de acceso se establece en el objeto de sesión para garantizar que solo los usuarios autenticados puedan acceder a los endpoints durante ese tiempo

The content of this lab is licensed under Apache 2.0

```js
// Login endpoint
app.post("/login", (req, res) => {
    const user = req.body.user;
    if (!user) {
        return res.status(404).json({ message: "Body Empty" });
    }
    // Generate JWT access token
    let accessToken = jwt.sign({
        data: user
    }, 'access', { expiresIn: 60 * 60 });

    // Store access token in session
    req.session.authorization = {
        accessToken
    }
    return res.status(200).send("User successfully logged in");
});
```

3. Observe la implementación del middleware de autenticación. Todos los puntos finales que comienzan con **/user** pasarán por este middleware. Recuperará los detalles de autorización de la sesión y los verificará. Si el token es validado, el usuario está autenticado y el control se pasa al siguiente manejador de punto final. Si el token es inválido, el usuario no está autenticado y se devuelve un mensaje de error.

The content of this lab is licensed under Apache 2.0

```js
// Middleware for user authentication
app.use("/user", (req, res, next) => {
    // Check if user is authenticated
    if (req.session.authorization) {
        let token = req.session.authorization['accessToken']; // Access Toker

        // Verify JWT token for user authentication
        jwt.verify(token, "access", (err, user) => {
            if (!err) {
                req.user = user; // Set authenticated user data on the reques
                next(); // Proceed to the next middleware
            } else {
                return res.status(403).json({ message: "User not authenticate
            }
        });

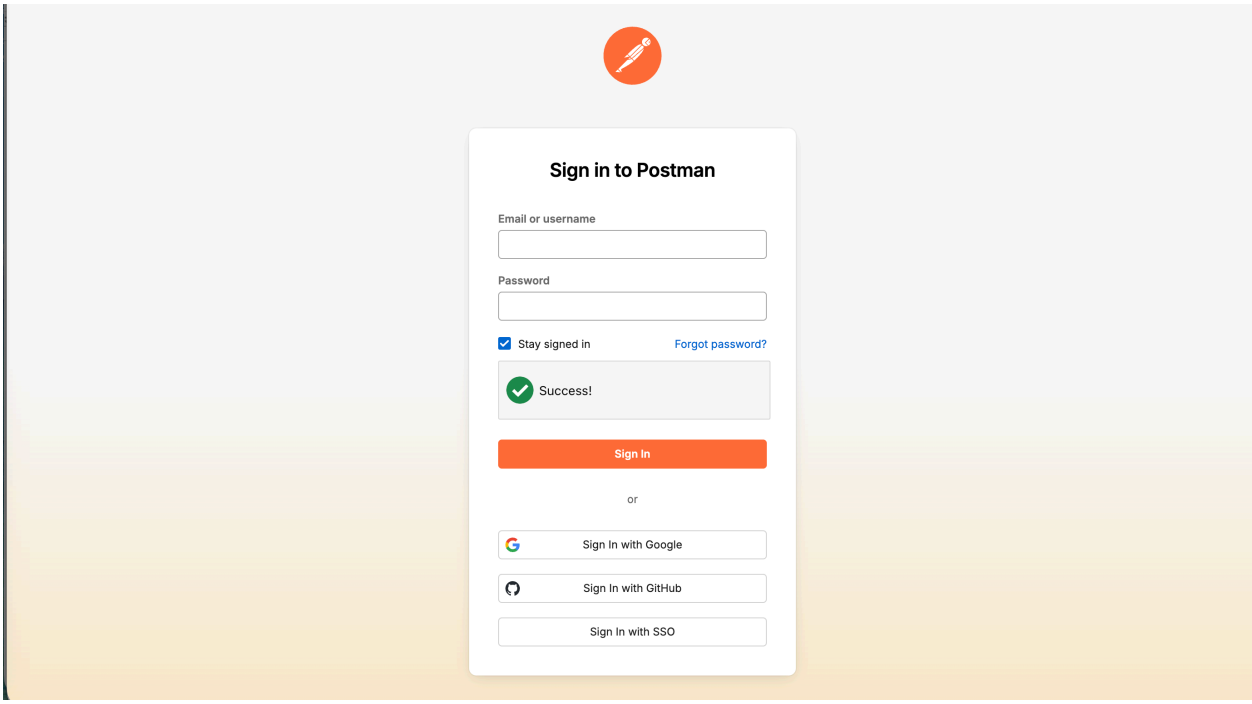
        // Return error if no access token is found in the session
    } else {
        return res.status(403).json({ message: "User not logged in" });
    }
});
```

The content of this lab is licensed under Apache 2.0

Ejercicio 8: Probando endpoints con POSTMAN

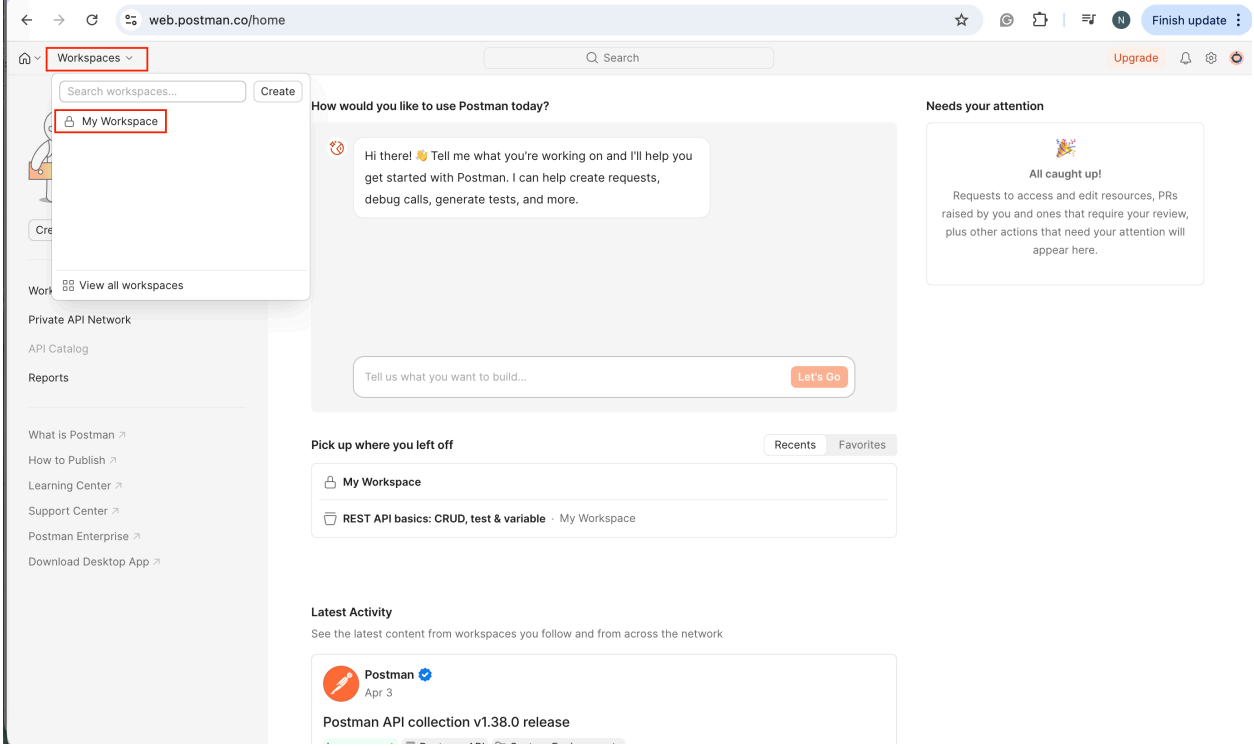
Has probado los endpoints de la API con cURL. Una forma más fácil y amigable de probar estos endpoints es con la herramienta de interfaz gráfica (GUI), Postman.

1. Abre Postman visitando: https://web.postman.co
   Si no tienes una cuenta, regístrate para obtener una nueva cuenta de Postman. De lo contrario, inicia sesión en tu cuenta existente.



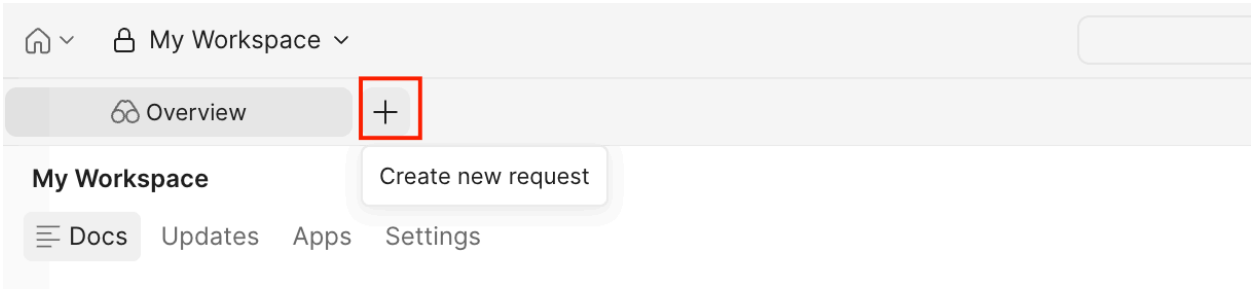
2. Después de iniciar sesión, navega a tu espacio de trabajo:
   - Haz clic en **Workspaces**
   - Selecciona **My Workspace**

The content of this lab is licensed under Apache 2.0



3. Crea una nueva solicitud:

    - Haz clic en el **"+" (icono de más)** para abrir una nueva pestaña de solicitud



> ***Nota:** Si el servidor está corriendo en el laboratorio theia, por favor detén el servidor presionando **CTRL + C**. Ahora inicia el servidor ejecutando el siguiente comando que escuchará en el puerto 5000.*

```sh
npm run start_auth
```

Run

> *Hasta ahora estábamos accediendo a todos los puntos finales sin autenticación, pero ahora usaremos autenticación para acceder a los puntos finales.*

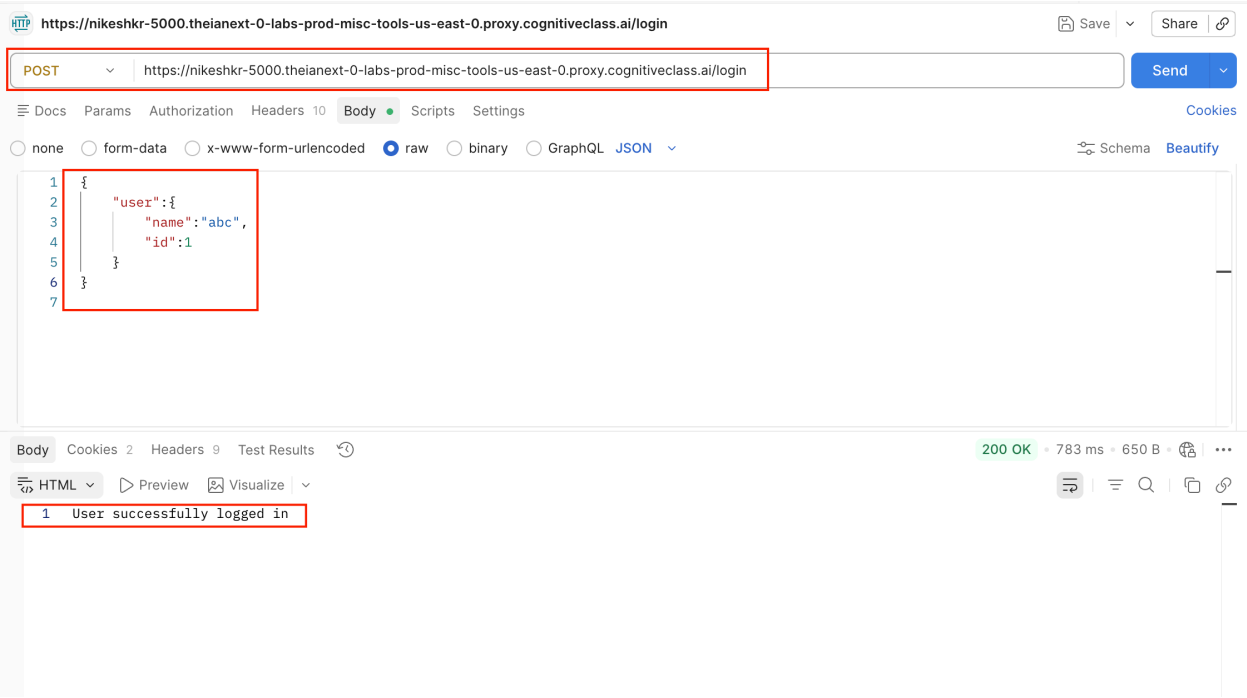
The content of this lab is licensed under Apache 2.0

4. Copia la URL de la aplicación de lanzamiento y añade el inicio de sesión como un punto final para agregar los detalles del usuario en la **SOLICITUD POST** que se verá como a continuación:

```plaintext
https://<sn-lab-username>-5000.theiadocker-2-labs-prod-theiak8s-4-tor01.proxy
```

5. Los detalles del usuario deben estar en el siguiente formato:

```json
{
    "user":{
        "name":"abc",
        "id":1
    }
}
```



Ahora comencemos la prueba enviando una solicitud HTTP GET.

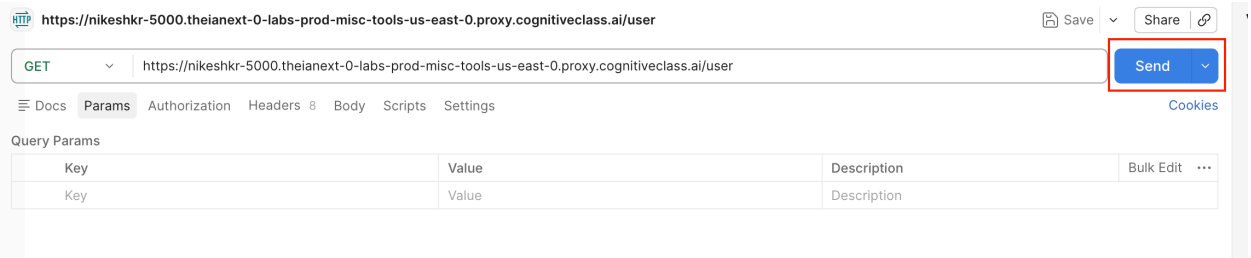
The content of this lab is licensed under Apache 2.0

8.1 Solicitud GET

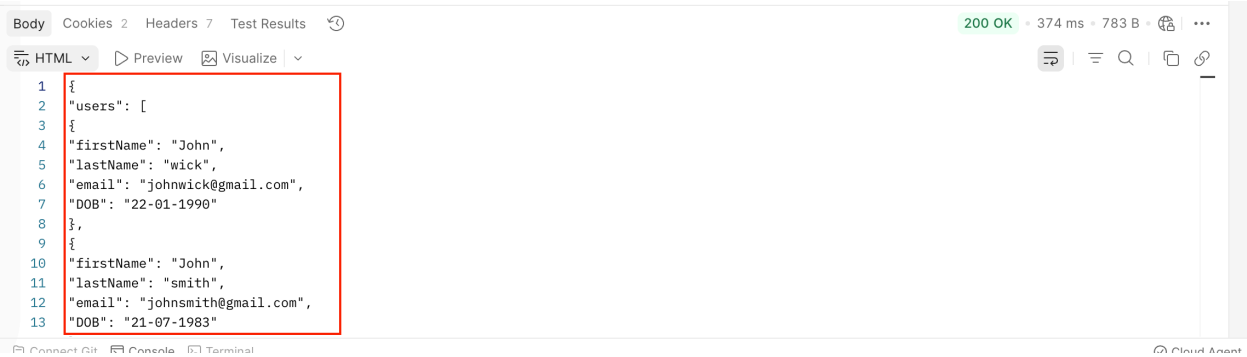
a. Introduzca la URL de la solicitud GET: `https://XXXXXXXXXX-5000.theiadocker-0-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai/user` en su cuadro de entrada de Postman donde dice "Introducir URL de solicitud".



b. Haga clic en el botón `Enviar` después de ingresar la URL.



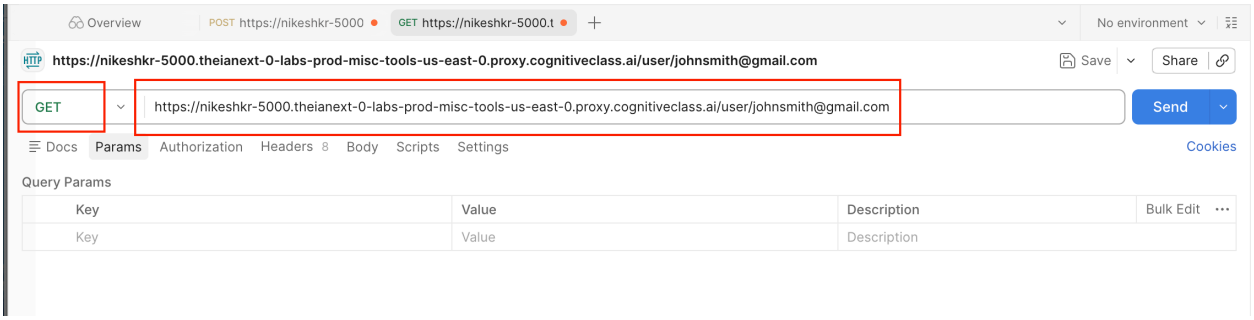
c. La salida será como se muestra a continuación:



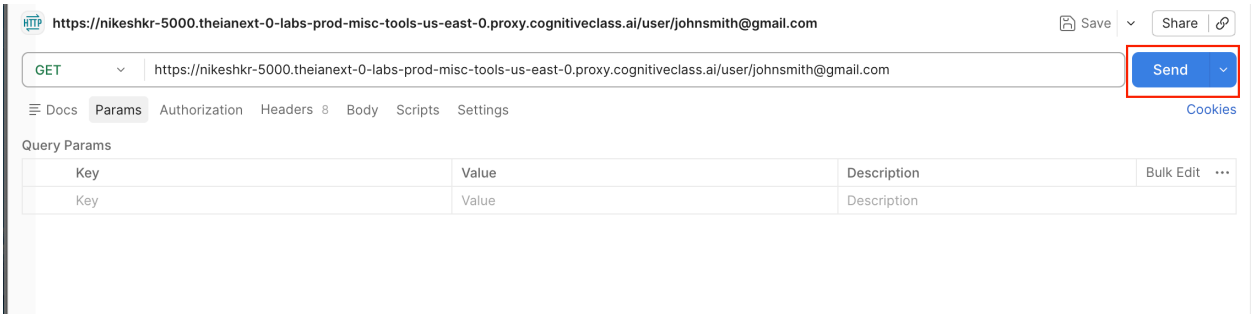
The content of this lab is licensed under Apache 2.0

8.2 Solicitud GET por ID específico

a. Ingresa la URL de la solicitud añadiendo la dirección de correo electrónico específica a la URL de la solicitud GET anterior. Si la dirección de correo electrónico es johnsmith@gmail.com, entonces ingresa la siguiente URL en el cuadro de entrada de Postman: `https://XXXXXXXXXX-5000.theiadocker-0-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai/user/johnsmith@gmail.com`



b. Haz clic en el botón **Enviar** después de ingresar la URL para ver la salida.



c. La salida será como se muestra a continuación:

The content of this lab is licensed under Apache 2.0



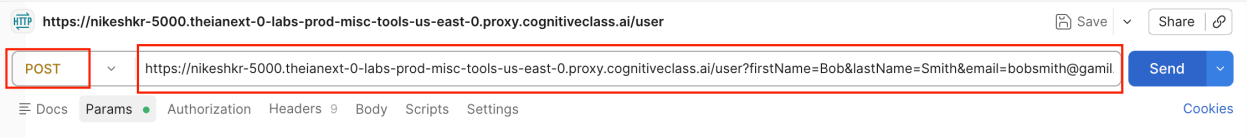
The content of this lab is licensed under Apache 2.0

8.3 Solicitud POST :

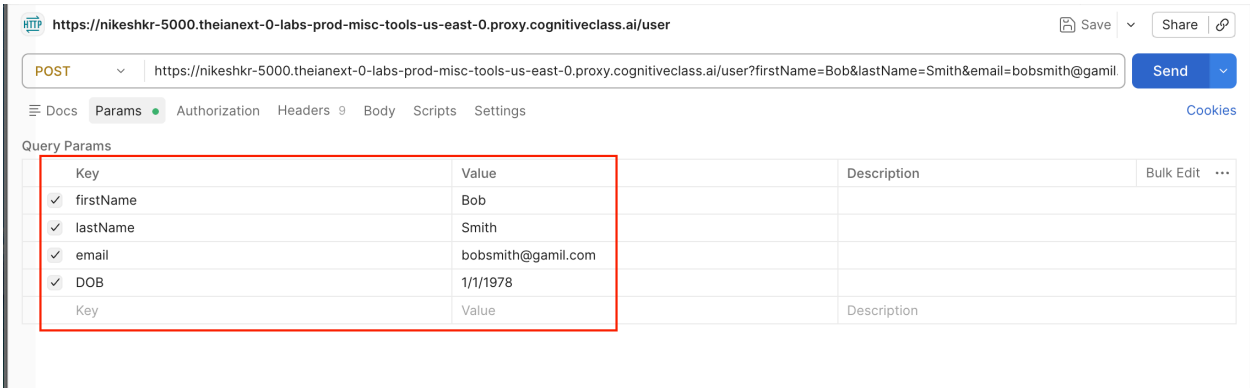
a. Ingresa la URL básica de la solicitud POST:

```plaintext
https://XXXXXXXXXX-5000.theiadocker-0-labs-prod-theiak8s-4-tor01.proxy.cognit
```

Asegúrate de seleccionar el método POST y seleccionar los "Params".

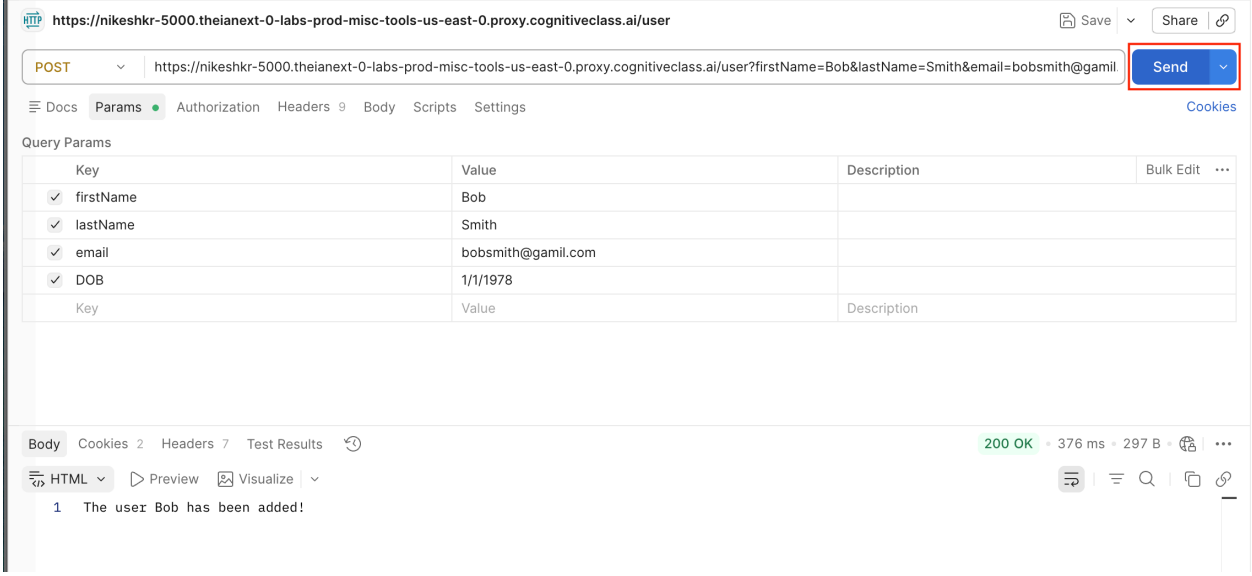


b. Ingresa el firstName como 'Bob', lastName como 'Smith', email como 'bobsmith@gmail.com' y DOB como '1/1/1978' para un nuevo usuario:



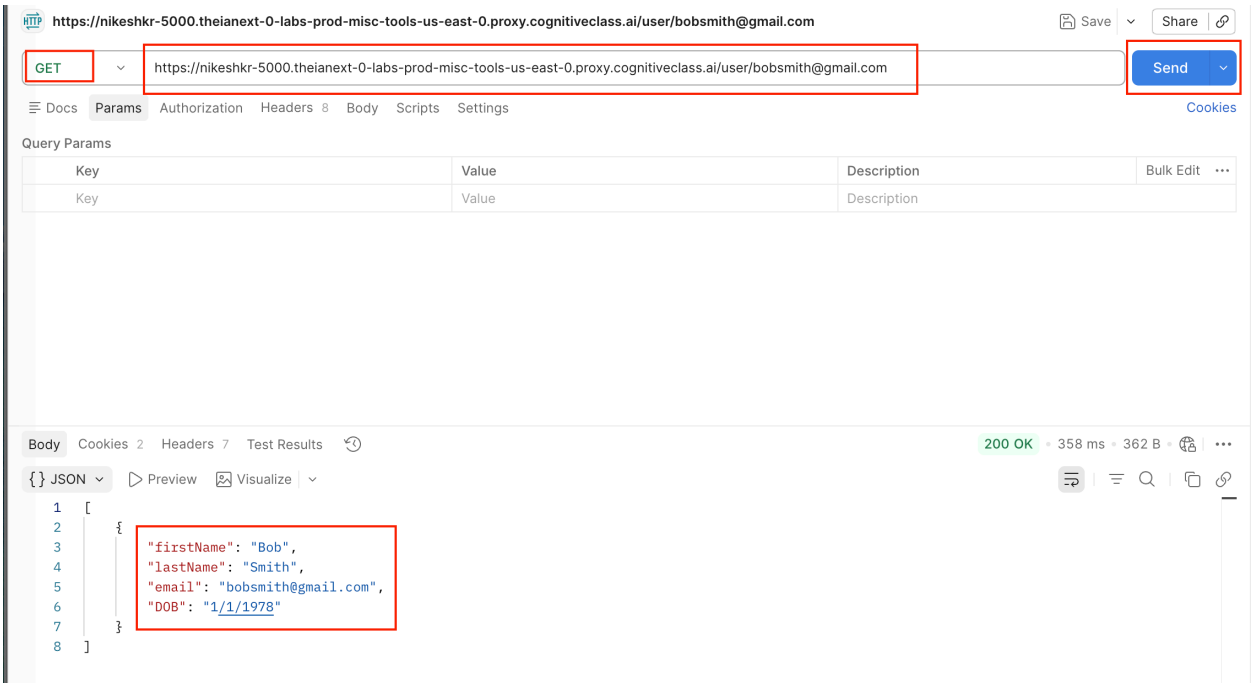
c. Haz clic en el botón **Send** después de ingresar la URL para ver la salida.

The content of this lab is licensed under Apache 2.0



Verifica que los nuevos valores añadidos se hayan actualizado realizando la solicitud GET.

> *Nota: Asegúrate de eliminar cualquier parámetro que hayas agregado para la solicitud POST antes de enviar la solicitud GET.*



The content of this lab is licensed under Apache 2.0

8.4 Solicitud PUT

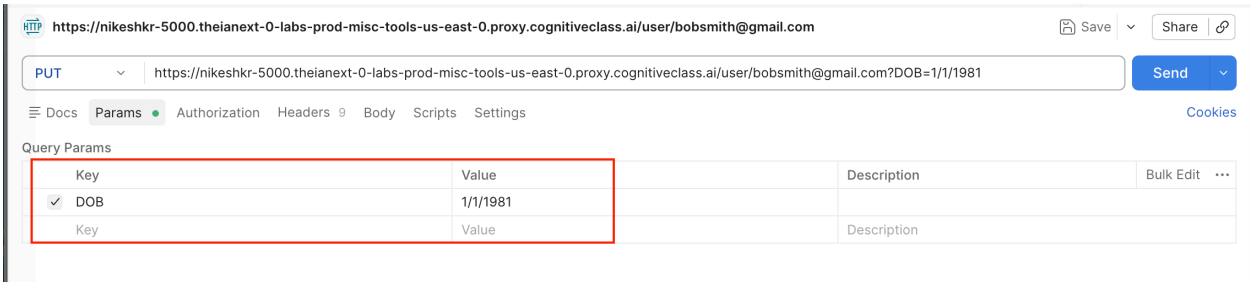
a. Ingresa la URL añadiendo la dirección de correo electrónico específica. Si la dirección de correo es bobsmith@gmail.com, entonces ingresa esta URL en el cuadro de entrada de Postman:

```plaintext
https://XXXXXXXXXX-5000.theiadocker-0-labs-prod-theiak8s-4-tor01.proxy.cognit
```

Asegúrate de seleccionar el método PUT y seleccionar los "Params".

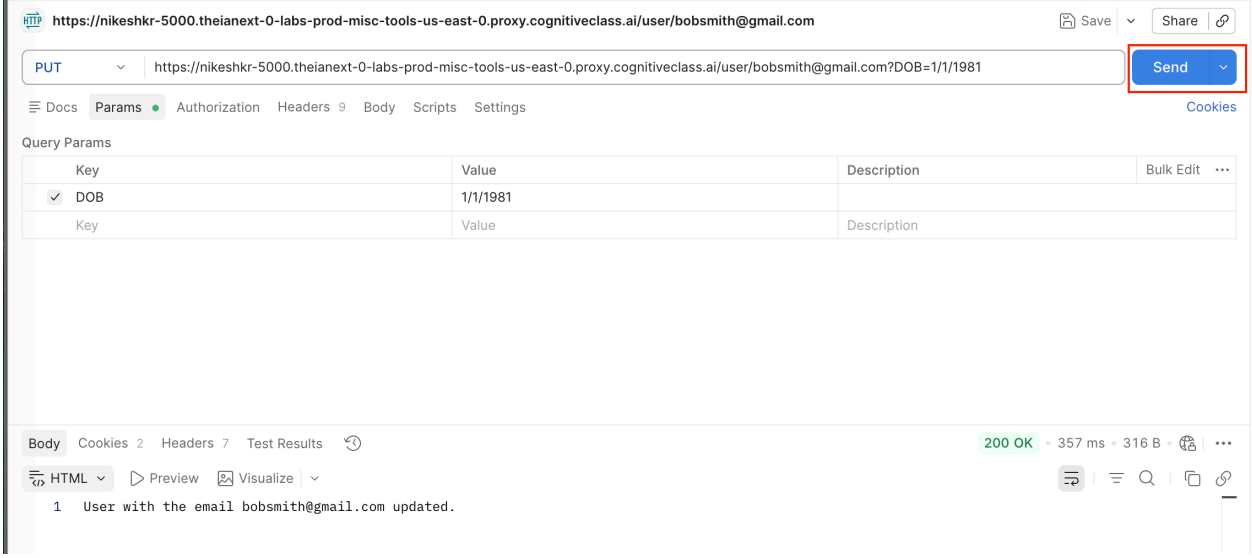


b. Ingresa la clave y los valores que se van a cambiar. Por ejemplo, si deseas cambiar la clave "DOB" y reemplazarla con el nuevo valor 1/1/1981, será como a continuación.



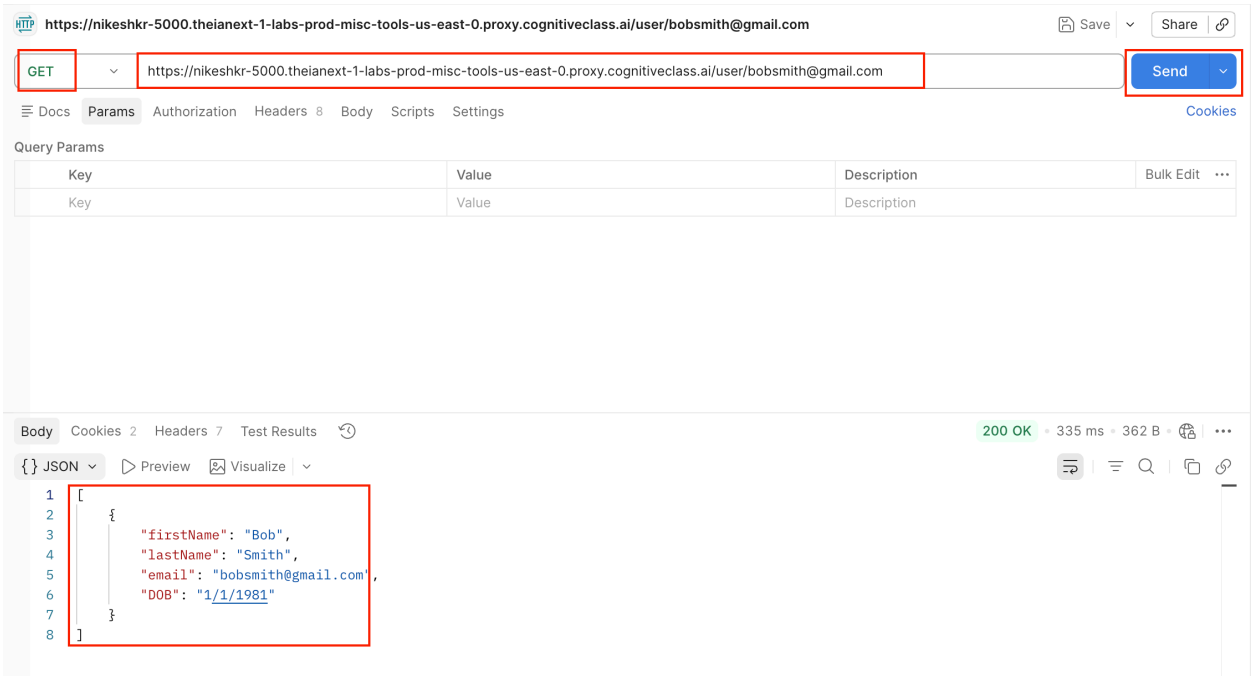
c. Haz clic en el botón **Send** después de ingresar la URL para ver la salida.

The content of this lab is licensed under Apache 2.0



Verifica que los nuevos valores añadidos se hayan actualizado realizando una solicitud GET.

> ***Nota: Asegúrate de eliminar cualquier parámetro que hayas añadido para la solicitud PUT antes de enviar la solicitud GET.***



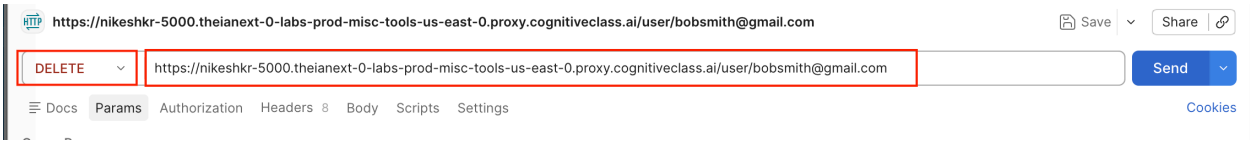
The content of this lab is licensed under Apache 2.0

8.5 Solicitud DELETE:

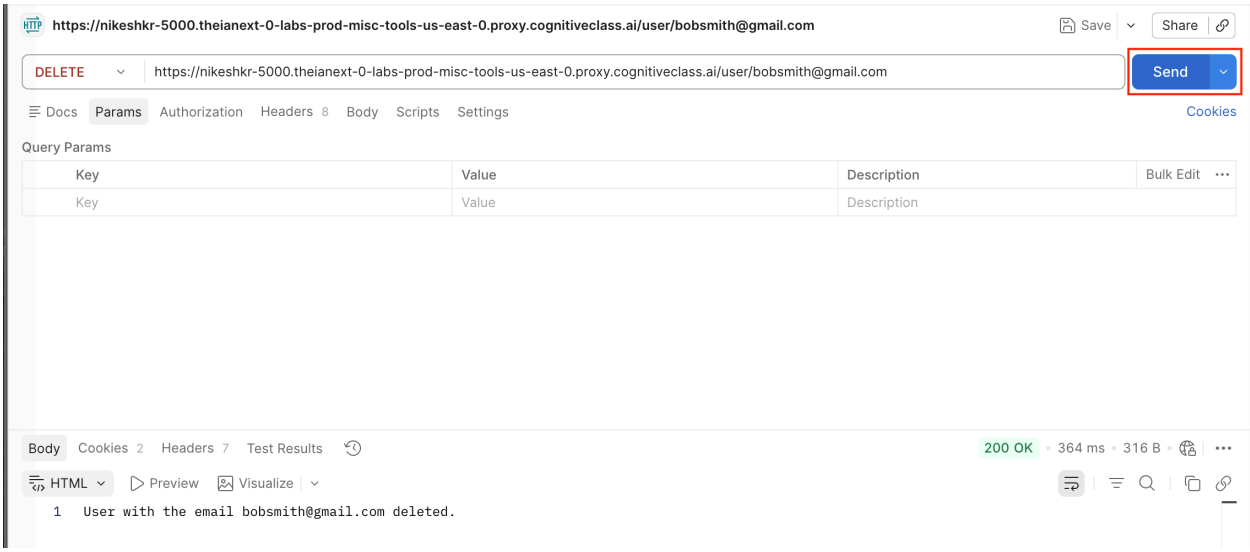
a. Ingresa la URL añadiendo la dirección de correo electrónico específica. Si la dirección de correo electrónico es bobsmith@gmail.com, entonces ingresa esta URL en el cuadro de entrada de Postman:

```plaintext
https://XXXXXXXXXX-5000.theiadocker-0-labs-prod-theiak8s-4-tor01.proxy.cognit
```

Asegúrate de seleccionar el método DELETE.



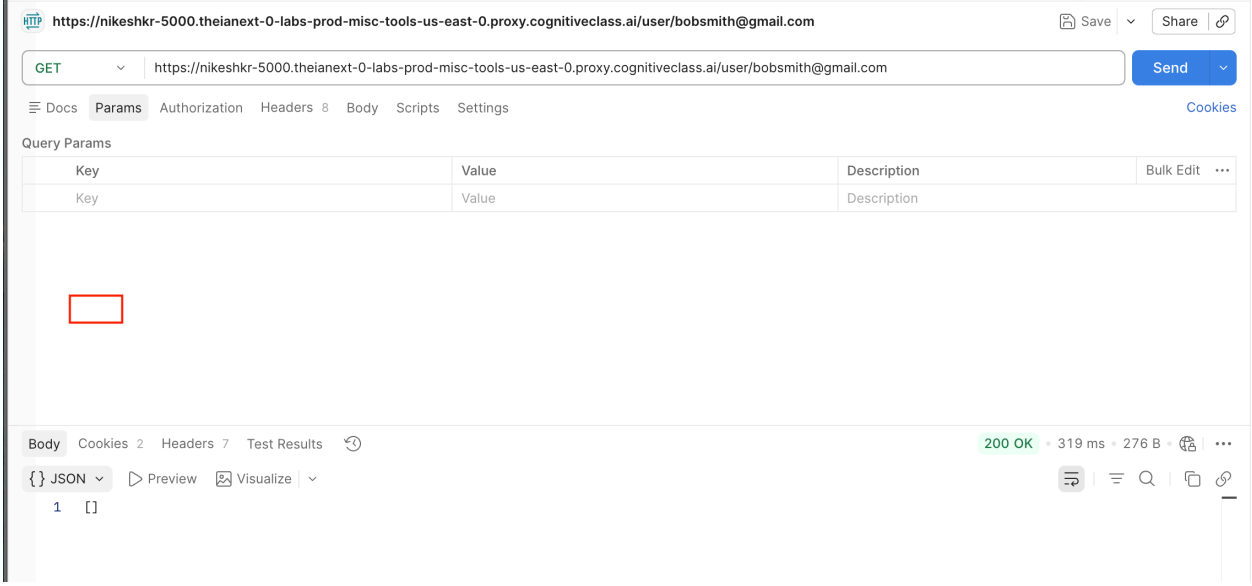
b. Haz clic en el botón "Enviar" después de ingresar la URL para ver la salida.



c. Verifica que el GET del usuario por ID bobsmith@gmail.com devuelve un objeto nulo enviando una solicitud GET.

> *Nota: Asegúrate de eliminar cualquier parámetro (si es que hay alguno) antes de*

The content of this lab is licensed under Apache 2.0



The content of this lab is licensed under Apache 2.0

Laboratorios de práctica

1. Crea un endpoint en el mismo código para obtener todos los usuarios con un apellido particular.

▶ ¡Haz clic aquí para una pista!

▶ ¡Haz clic aquí para la solución!

2. Crea un endpoint en el mismo código para ordenar usuarios por fecha de nacimiento.

▶ Haz clic aquí para una pista!

▶ Haz clic aquí para la Solución!

¡Felicidades! Has completado el laboratorio de operaciones CRUD con Node.js y Express.js utilizando Postman.

Resumen:

En este laboratorio, hemos realizado operaciones CRUD como GET, POST, PUT y DELETE en una aplicación Express y hemos probado los métodos anteriores utilizando Postman.

The content of this lab is licensed under Apache 2.0

Author(s)

Sapthashree K S

K Sundararajan

© IBM Corporation. Todos los derechos reservados.

Laboratorio práctico - Operaciones CRUD con Node.js



Tiempo estimado necesario: 1 hora

En este laboratorio aprenderás a crear una **Lista de amigos** utilizando un servidor Express. Tu aplicación debe permitirte agregar un amigo con los siguientes detalles: `First name`, `Last name`, `Email` y `Date of birth`. También proporcionarás a la aplicación la capacidad de recuperar detalles, cambiar detalles y eliminar los detalles.

Estarás creando una aplicación con puntos finales de API para realizar operaciones de Crear, Recuperar, Actualizar y Eliminar sobre los datos mencionados utilizando un servidor Express.

También aprenderás a proporcionar acceso autenticado a los puntos finales. Utilizarás cURL y Postman para probar los puntos finales implementados.

The content of this lab is licensed under Apache 2.0

Objetivos:

- Crear puntos finales de API para realizar operaciones de Crear, Recuperar, Actualizar y Eliminar sobre datos transitorios con un servidor Express.
- Implementar autenticación a nivel de sesión utilizando JSON Web Tokens (JWT) para acceso autorizado.

Configuración : Crear apli… →

The content of this lab is licensed under Apache 2.0